CPE 314 — Software Design & Software Design Laboratory

Week 8 — Project Proposal

Due end of Week 8 session | 10% of the Finals Term Grade | Scored out of 100

Group Name / No.:	PNP/1
Members:	Kimberly Luistro (Team Leader), Nesley Cate Librea (UI), Yvonne Mae Tapan (UI), Venice Nadela (QA), Kenneth Tamidles (Database Admin), Kirk Ryan Quisagan (Developer)
Repository URL:	https://github.com/Kimonooo/cpe314-69167-capstoneteam-Meal_plannerGroup1
Chosen Theme:	A B C D E F G **H** I J K L M N O P Q R S T — **Option H: Recipe & Meal Planner**
API You Will Use:	**TheMealDB —** https://www.themealdb.com/api.php (shared free test key "1", no sign-up or login). Endpoint: GET https://www.themealdb.com/api/json/v1/1/search.php?s={name}

1. One-Paragraph Pitch

Plate N Platter is a personal meal planner for students and busy households who struggle to decide what to cook, what to buy, and whether they are eating well. Each person signs up and logs in to a private account, searches real recipes from TheMealDB, and plans them on a calendar as Breakfast, Lunch, Dinner or Snack. Every planned meal carries its calories and a Healthy Level (Healthy, Moderate or Indulgent) that the app works out from the calories and that meal slot's share of a 2,000-calorie day, and each meal is tagged Quick, Average or Slow by its prep time. Recipes can contain other recipes, and the app adds up prep time and calories through every level. A shopping list merges the ingredients of the planned meals and groups them by store aisle, a planning streak earns a badge (Quick Cook, Home Cook or Slow-Food Chef), years of meal history can be imported from a CSV file without freezing the window, and an Analytics tab turns the plan into a grouped bar chart of meals per day.

2. Entity-Relationship Sketch

Figure 1 shows the eight entities that our SQLite database creates in src/main/resources/schema.sql, with their key attributes and the cardinality of every relationship. The sketch now matches the schema table for table, and Tables 1 and 2 list the same entities, attributes, keys and relationships in text.

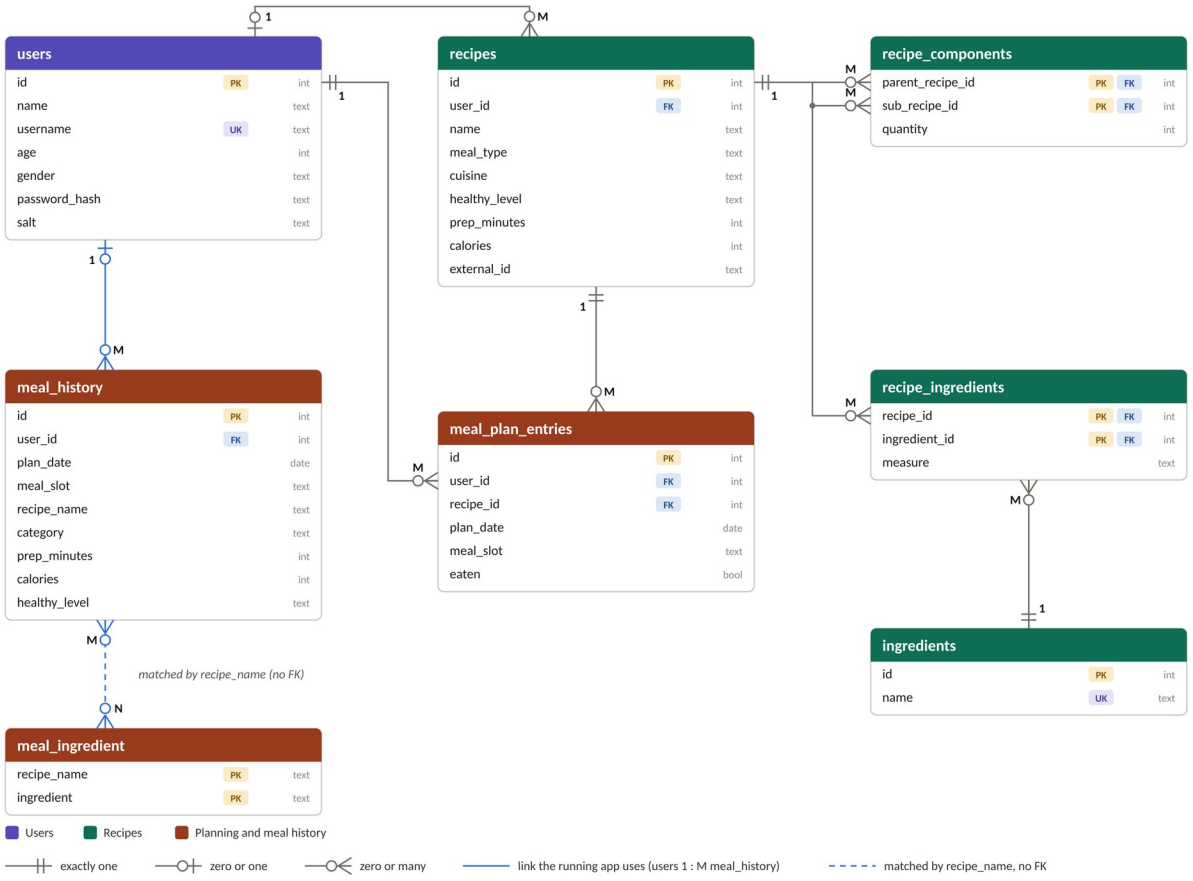


Figure 1. Entity-Relationship sketch (8 entities, matching schema.sql)

Table 1. Entities, key attributes, keys and cardinality

Entity	Key attributes	PK / FK / UK	Cardinality
users	name, username, age, gender, password_hash, salt	PK: id UK: username	1 user : M recipes 1 user : M meal_plan_entries 1 user : M meal_history
recipes	name, meal_type, cuisine, healthy_level, prep_minutes, calories, external_id	PK: id FK: user_id → users.id	M recipes : 1 user 1 recipe : M meal_plan_entries 1 recipe : M recipe_components 1 recipe : M recipe_ingredients

Entity	Key attributes	PK / FK / UK	Cardinality
recipe_components	quantity	PK: parent_recipe_id + sub_recipe_id FK: parent_recipe_id → recipes.id FK: sub_recipe_id → recipes.id	Join table for recipes M : N recipes (a recipe inside a recipe)
ingredients	name	PK: id UK: name	1 ingredient : M recipe_ingredients
recipe_ingredients	measure	PK: recipe_id + ingredient_id FK: recipe_id → recipes.id FK: ingredient_id → ingredients.id	Join table for recipes M : N ingredients
meal_plan_entries	plan_date, meal_slot, eaten	PK: id FK: user_id → users.id FK: recipe_id → recipes.id	M entries : 1 user M entries : 1 recipe
meal_history	plan_date, meal_slot, recipe_name, category, prep_minutes, calories, healthy_level	PK: id FK: user_id → users.id	M meals : 1 user (the link every tab uses)
meal_ingredient	recipe_name, ingredient	PK: recipe_name + ingredient No FK	M : N meal_history, matched by recipe_name

Table 2. Relationships and cardinality

From	To	Cardinality	Through and meaning
users	recipes	1 : M	recipes.user_id: a user saves many recipes
users	meal_plan_entries	1 : M	meal_plan_entries.user_id: a user plans many recipe entries
users	meal_history	1 : M	meal_history.user_id: a user owns many planned or imported meals
recipes	meal_plan_entries	1 : M	meal_plan_entries.recipe_id: a recipe can be planned many times
recipes	recipes	M : N	recipe_components: a recipe uses other recipes as sub-recipes
recipes	ingredients	M : N	recipe_ingredients: a recipe has many ingredients and an ingredient is in many recipes

From	To	Cardinality	Through and meaning
meal_history	meal_ingredient	M : N	recipe_name (no foreign key): a planned dish finds its ingredient list by name

Change after the Week 8 review. The first sketch also drew Streak, UserBadge, ShoppingListItem, CookQueueEntry and Favorite, which the schema never created. They were removed: the streak and badge are worked out from the meal dates, the shopping list is built for a date range when it is opened, the cook queue and undo stack live in memory, and Favorites was a stretch goal. meal_history and meal_ingredient were added because every tab of the app reads and writes them.

3. Feature Checklist

Check off which mandatory requirement each planned feature will satisfy:

Planned Feature	Requirement(s) It Satisfies
Sign up (name, username, age, gender, password, confirm password) and Log in with salted PBKDF2 password hashes and clear error messages; Account tab to change the display name	Database (JDBC PreparedStatement), Event-Driven GUI
Abstract Recipe with BreakfastRecipe, MainCourseRecipe and DessertRecipe, each with its own prep-time rule (+0, +10, +15 minutes); PlanStore interface with a SQLite and an in-memory implementation	Object-Oriented Design
Recursive totals of prep time and calories across nested sub-recipes, with protection against circular references; the total prep time sets Quick / Easy / Hard	Recursion
Cooking queue: recipes come out first in, first out (CookingQueue)	Queue
Undo stack: the most recent change is undone first (UndoStack)	Stack
Shopping list that merges the ingredients of the planned meals (ingredient → number of meals) and groups them by store aisle	Map
Meal Plan calendar grid; each day is colored by the Healthy Level most of its meals have; every meal row shows its slot, health chip, prep tier and calories	Event-Driven GUI, Database
Today tab with Breakfast, Lunch, Dinner and optional Snack, each with an Add button; live date, clock, greeting and streak badge in the header	Event-Driven GUI
Recipes tab: search TheMealDB by name, pick the meal slot, type the calories, keep Healthy Level on Auto or choose one, and add the meal to the plan	API Consumption, Event-Driven GUI (3+ controls), Database

Planned Feature	Requirement(s) It Satisfies
Parse the TheMealDB JSON (meals array, strIngredient1 to 20) into MealData objects and map its categories to Breakfast, Main Course or Dessert	API Consumption
CSV import of meal-plan history (3,000+ rows), TheMealDB search, log-in hashing and chart loading run on background Tasks with a progress bar and Cancel	Concurrency
Statistics: meals per day grouped by Healthy Level, prep tier or meal slot; healthy share; average prep time; average calories	Data Mining
Grouped bar chart “Meals Planned per Day by …” with labeled axes and a legend, for one week or all time	Data Visualization
Planning streak (days in a row with a planned meal) and a badge named after the prep tier cooked most: Quick Cook, Home Cook or Slow-Food Chef	Data Mining (consecutive-day analysis), Event-Driven GUI
Each user sees only their own meals (user_id foreign key; one data-access object per logged-in user)	Database
6+ JUnit 5 tests (185 at Milestone 2): password hashing, recursion (plain recipe, one sub-recipe, 3-level nest, cycles), queue order, undo stack, shopping-list merge, streak counting, CSV bad rows, DAO with a temporary SQLite file	Automated Testing

4. Task Split

Member	Primary Responsibility (this milestone)
Kimberly Luistro	**Team Leader:** • Builds the Recipe classes (BreakfastRecipe, MainCourseRecipe, DessertRecipe), MealPlanEntry, the cooking queue and the undo stack. • Builds the app start-up, the tab shell and the Analytics tab with the grouped chart. • Sets milestones, reviews and merges pull requests, and handles submissions.
Nesley Cate Librea	**UI:** • Builds the Log in, Sign up and Account screens, the Recipes tab (TheMealDB search, meal boxes) and the Meal Plan calendar. • Writes the shared maroon-and-cream stylesheet.
Yvonne Mae Tapan	**UI:** • Builds the live header (date, clock, streak badge), the Today tab grouped by meal slot and the Import tab with its progress bar. • Writes the streak calculation and the prep-tier badge.

Member	Primary Responsibility (this milestone)
Venice Nadela	**QA:** • Writes the JUnit 5 tests and keeps a manual test checklist per screen. • Tests log-in errors, circular references, CSV bad rows, failed API calls and per-user data; keeps the known-issues list and tracks bugs.
Kenneth Tamidles	**Database Admin:** • Creates the SQLite tables (schema.sql) and the JDBC data-access classes (UserDao, MealPlanHistoryDao) with PreparedStatement. • Stores salted password hashes, keeps each user's meals separate and upgrades old database files.
Kirk Ryan Quisagan	**Developer:** • Writes the recursive prep-time and calorie totals with circular-reference protection. • Writes the meal slot, Healthy Level and prep tier rules, the shopping list grouped by aisle and the statistics used by the chart.

5. Public API You Plan to Use

TheMealDB (https://www.themealdb.com/api.php, endpoints under https://www.themealdb.com/api/json/v1/1/) is a free public REST API of real recipes that Java can call with no login wall; the shared test key 1 is part of the URL. We call search by name (search.php?s={name}) with Java's HttpClient. Each result gives the meal's name, category, cuisine (area), step-by-step instructions and up to twenty ingredients with measures. MealDbClient parses the JSON with Gson into MealData objects; the Recipes tab lists and shows them, the chosen meal is saved with its calories and Healthy Level, its ingredients are saved for the Shopping List, and the planned meal feeds the streak and the chart. We checked the endpoint against the live documentation page and in a browser, where the raw JSON can be shown during the demo.

6. Risks / Open Questions

Each risk below has our plan and the question we raised with the instructor. The last column records the instructor's response from the review.

Risk or open question	Our plan	Instructor response
TheMealDB has no calories, nutrition or prep-time data. Is user-entered calories with an automatic Healthy Level acceptable instead of a second (nutrition) API?	The user types the calories and prep minutes. HealthClassifier sets the Auto level from the calories and the meal slot's budget (Breakfast 500, Lunch 650, Dinner 650, Snack 200 kcal); the user can override it. A second API was dropped to keep one free API with no key to protect.	

Risk or open question	Our plan	Instructor response
TheMealDB categories (Beef, Chicken, Seafood…) differ from Breakfast / MainCourse / Dessert. Is our mapping rule acceptable?	CategoryMapper keeps Breakfast and Dessert and maps every other category (or a blank one) to Main Course; it is covered by CategoryMapperTest.	
Sub-recipes are not provided by the API, so a user could make a recipe that contains itself.	RecipeCalculator.wouldCreateCycle checks before a sub-recipe is added, and the recursive totals throw CircularRecipeException if a loop is met; both cases are tested.	
Are our prep-time thresholds acceptable?	Nested-recipe totals use Quick / Easy / Hard (up to 30 / 31–60 / over 60 min). Planned meals use PrepTier Quick / Average / Slow (up to 15 / 16–45 / over 45 min), which also names the streak badge.	
Database: the spec requires SQLite via JDBC with PreparedStatement. Is local SQLite enough without the Cloudflare D1 cloud sync we first proposed?	One local file, platenplatter.db, is the only database; D1 sync was dropped so no API token is stored in the desktop app. Foreign keys are switched on for every connection.	
Is a flat meal_history table (planned and imported meals) acceptable next to the normalized recipe tables?	meal_history lets the import, the calendar, the streak and the chart share one table and one query; the normalized recipe tables stay in schema.sql for the full recipe model.	